

■ Lab 8 — Expressions Conditionnelles (AWS CloudShell)

■ Objectifs du Lab

- Utiliser `condition ? true_val : false_val`
- Adapter l'infrastructure selon l'environnement **dev / prod**

■ Étape 1 — Créer les fichiers

```
bash $HOME/install-terraform.sh
cd $HOME/terraform-training/lab08-conditionals
```

```
cat > backend.tf << 'EOF'
terraform {
  backend "s3" {
    bucket      = "tf-training-<votre-prenom>-982908300187"
    key         = "terraform.tfstate"
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}
EOF

cat > versions.tf << 'EOF'
terraform {
  required_version = ">= 1.15.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}
EOF

cat > provider.tf << 'EOF'
provider "aws" { region = var.region }
EOF

cat > variables.tf << 'EOF'
variable "username" { type = string }
variable "region" { type = string; default = "eu-west-1" }
variable "environment" {
  type = string
}
```

```
default = "dev"
validation {
  condition      = contains(["dev", "prod"], var.environment)
  error_message = "L'environnement doit être dev ou prod."
}
}
EOF

cat > locals.tf << 'EOF'
locals {
  prefix          = "lab08-${var.username}-${var.environment}"
  instance_type   = var.environment == "prod" ? "t2.small" : "t2.micro"
  instance_count  = var.environment == "prod" ? 2 : 1
  monitoring_enabled = var.environment == "prod" ? true : false
  common_tags = {
    Lab = "lab8", Username = var.username, Environment = var.environment, ManagedBy = "Terraform"
  }
}
EOF

cat > main.tf << 'EOF'
resource "aws_security_group" "lab8_sg" {
  name          = "${local.prefix}-sg"
  description   = "Security group Lab 8 - ${var.username} - ${var.environment}"
  tags          = merge(local.common_tags, { Name = "${local.prefix}-sg" })
}

resource "aws_instance" "lab8_instance" {
  count          = local.instance_count
  ami            = "ami-0694d931cee176e7d"

  instance_type   = local.instance_type
  monitoring      = local.monitoring_enabled
  vpc_security_group_ids = [aws_security_group.lab8_sg.id]
  tags = merge(local.common_tags, { Name = "${local.prefix}-instance-${count.index + 1}" })
}
EOF

cat > outputs.tf << 'EOF'
output "instance_ids"          { value = aws_instance.lab8_instance[*].id }
output "instance_type_used"    { value = local.instance_type }
output "instance_count_used"   { value = local.instance_count }
output "monitoring_enabled"    { value = local.monitoring_enabled }
EOF
```

■ Étape 2 — Déployer en dev

```
terraform init
terraform apply -var="username=<votre-prenom>" -var="environment=dev"
terraform output instance_type_used
terraform output instance_count_used
terraform output monitoring_enabled
```

■ Étape 3 — Déployer en prod

```
terraform apply -var="username=<votre-prenom>" -var="environment=prod"
terraform output instance_type_used
terraform output instance_count_used
terraform output monitoring_enabled
```

■ Étape 4 — terraform console

```
terraform console -var="username=<votre-prenom>" -var="environment=prod"
```

```
> local.instance_type
> local.instance_count
> local.monitoring_enabled
```

Tapez `exit` pour quitter.

■ Étape 5 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>" -var="environment=prod"
```

■ Validation du Lab

#	Critère	Validé
1	`instance_type_used` retourne `t2.micro` en dev	■
2	`instance_type_used` retourne `t2.small` en prod	■

#	Critère	Validé
3	`instance_count_used` retourne `1` en dev et `2` en prod	■
4	`monitoring_enabled` retourne `false` en dev et `true` en prod	■
5	`terraform destroy` supprime toutes les ressources	■

■ ****Fin du Lab 8 !****